

CPS4005 - Object-Oriented Programming - Assessment Brief

Module Code:	CPS4005
Module Title:	Object-Oriented Programming
Module Convenor:	Harshil Joshi
Module Level:	4

Assessment Number:	2
Assessment Title:	St Mary's Digital Library System
Assessment Weight:	60%
Assessment Individual/Group:	Individual
Assessment Type:	Software Artefact with Report
Assessment Time/Word Count Restrictions:	1 software artefact 1800 words report
Assessment Time/Word Count Limit Consequences:	It is essential that assignments keep within the time/word count limit stated above. Any work beyond the maximum time/word length permitted will be disregarded and not accounted for in the final grade.

Issue Date:	March 9, 2026
Hand in Date:	May 7, 2026
Planned Feedback Date:	Within 3 working weeks
Mode of Submission:	Online via Moodle
Number of copies to be submitted:	1 copy of each of the following: - a report in pdf format - a zip file containing your software artefact

Author:	Harshil Joshi
Internal Moderator:	Prins Butt
Moderation Date:	March 9, 2026

CPS4005 - Object-Oriented Programming - Assessment Brief

Table of Contents

Introduction	3
Problem Scenario	3
Requirements	4
Environment and Tools	8
Submission	9
Assessment Criteria	10
Additional Artefacts and Guidance	11
Learning Outcomes	14
Regulations, Policies, and Guidelines	16

CPS4005 - Object-Oriented Programming - Assessment Brief

Introduction

This assessment aims to evaluate your ability to apply object-oriented programming concepts and technologies in a real-world scenario. You are tasked with designing and implementing a robust object-oriented solution for a fictional firm, St Mary's Digital Library System.

Problem Scenario

St Mary's University Library serves thousands of students and staff members who borrow books, reserve study materials, and track return deadlines. Currently, the library relies on outdated manual processes to manage books, borrowing records, and returns. This has resulted in several issues:

- Difficulty tracking which books are currently borrowed
- Limited ability to monitor overdue books
- Inefficient management of library inventory

To address these challenges, the university has decided to develop a Digital Library Management System.

As a software developer, your task is to design and implement a Java-based application that helps the library staff manage books, members, and borrowing activities.

The system must:

- Manage Books - store and update book details in the library collection.
- Manage Library Members - maintain student or staff borrowing records.
- Track Borrowing Transactions - record book loans, returns, and due dates.

CPS4005 - Object-Oriented Programming - Assessment Brief

Requirements

You are required to develop a Java-based library management system for St Mary's University. The application must interact with the provided SQLite database and satisfy the following detailed requirements:

Basic Requirements (Minimum Viable Product)

1. CRUD Operations:

- **Book Management**
 - Create: Add new books with details such as Book ID, Title, Author, Category, and Availability Status.
 - Retrieve: Display all books in the library, Search books by title, author, or ID.
 - Update: Modify book information such as title, category, or availability status.
 - Delete: Remove books from the library database with confirmation prompts.
- **Member Management**
 - Create: Register new members with details such as Member ID, Name, Email, and Membership Type.
 - Retrieve: Display member information, Search members by name or ID.
 - Update: Modify member details.
 - Delete: Remove members from the system when required.
- **Borrowing Management**
 - Create: Record new borrowing transactions including Book ID, Member ID, Borrow Date, and Due Date.
 - Retrieve: Display borrowing history, Search records by member or book.
 - Update: Update borrowing status (Returned, Overdue, Borrowed).
 - Delete: Remove incorrect borrowing records.

2. Console-Based User Interface:

CPS4005 - Object-Oriented Programming - Assessment Brief

- Develop a text-based, menu-driven console application that allows library staff to interact with the system.
- Example menu options:
 - Manage Books
 - Manage Members
 - Manage Borrowing Records
 - Search Records
 - Exit System
- The interface should guide users clearly through input processes and display meaningful system messages.

3. Database Integration:

- Your application must connect to an SQLite database.
- Requirements:
 - Use SQLite JDBC for database connectivity
 - Store all book, member, and borrowing data in the database
 - Ensure CRUD operations interact directly with the database
 - Maintain persistent data storage

Medium Requirements (Enhanced Features)

1. Graphical User Interface (GUI):

- **Framework:** Develop the interface using Java Swing or JavaFX.
- **Requirements:**
 - Main Window: Dashboard displaying Books, Members, and Borrowing Records.
 - Navigation: Tabs or menu system for accessing each module.
 - Forms: Forms for adding, updating, or deleting records.
 - Controls: Buttons such as Add, Update, Delete, and Refresh.
 - Usability: Clear layout, labels and Confirmation dialogues for important actions

2. Enhanced Data Validation and Error Handling:

- Implement the following validation mechanisms to ensure data integrity.
- Input Validation:
 - Book IDs must be numeric
 - Email addresses must follow the correct format
 - Borrow dates must be valid
- Error Handling:
 - Handle invalid input gracefully
 - Handle database connection failures
 - Display meaningful error messages

CPS4005 - Object-Oriented Programming - Assessment Brief

- User Feedback: Provide confirmation messages such as:
 - “Book added successfully”
 - “Borrow record updated successfully”
- 3. **Data Visualization and Interaction:**
 - Enhance the usability of the system.
 - Display records using tables
 - Enable sorting of data
 - Allow filtering by category, author, or member
 - Implement search functionality

Advanced Requirements (Performance and Scalability)

1. **Multi-Threading and Concurrency:**
 - Implement multi-threading to handle background tasks such as:
 - Loading data from the database without freezing the GUI.
 - Performing batch updates or concurrent operations (e.g., simultaneous inventory updates and order processing).
 - Utilize Java concurrency mechanisms to manage background processes.
 - Ensure proper synchronization and thread safety to prevent data conflicts or corruption when multiple threads access shared resources.
2. **Advanced Search and Filtering:**
 - Develop advanced search capabilities. For Examples,
 - Search books by multiple attributes (title, author, category)
 - Filter borrowing records by date range
 - Identify overdue books automatically
 - Allow sorting results in ascending or descending order.
3. **Performance Optimization:**
 - Ensure the system can handle large datasets efficiently. For Examples,
 - Optimize database queries
 - Use asynchronous data loading
 - Provide progress indicators for long-running operations

By addressing these detailed requirements, your application will evolve from a basic console-based system to a fully featured, interactive, and scalable solution that effectively supports library operations. This comprehensive approach ensures all aspects of functionality, usability, and performance are met.

CPS4005 - Object-Oriented Programming - Assessment Brief

Environment and Tools

You are required to use any of the following tools:

- **Adobe PDF Reader**: To view your final report which is saved as a PDF file.
- **Draw.io** (or equivalent tool): To create diagrams.
- **Git Tools** and **GitHub** for version control.
- **Microsoft Word** (or equivalent tool): To write your report and export as a PDF.
- **NetBeans, VSCode or JDoodle** as your integrated development environment.
- **Java and JDK22+** as the standard programming language and development kit.
- **SQLite 3** as your database management system.
- **SQLite Studio** (or equivalent tool) to browse your database.

- Additionally, the following libraries may be utilised:

Standard Java Libraries - these are:

java.io,
java.util,
java.nio,
java.sql,
java.time,
java.util.concurrent
java.util.logging

SQLite JDBC - for database connectivity

Java Collections Framework - for managing collections

JavaFX - for graphical interface development

Java Swing - for graphical interface development

Log4j - for logging capabilities

Junit - for unit testing

Mockito - to create mock objects and simulate dependencies in unit tests

- No other Java libraries should be used

CPS4005 - Object-Oriented Programming - Assessment Brief

Submission

The assessment must be completed individually. You must not share, in part or whole, your assessment with another party other than the module convenor and for the purpose of submission to the university. You must ensure that the University's academic misconduct guidelines are followed in their entirety.

You should use the assessment submission link on the module's Moodle page to submit the following files:

- A **PDF** file for your report. This should not be included in the zip file but instead submitted as a separate file. Failure to do so may result in zero being awarded.
- A **Zip** file of your software artefact. This should contain your solution files, git log, and other files required to view and run your solution.

You should ensure that you make a timely submission by the deadline stated at the start of this assessment brief.

CPS4005 - Object-Oriented Programming - Assessment Brief

Assessment Criteria

Your assessment will be graded according to the following criteria:

Grading criteria	Functionality	Documentation	Professional Practice
Mark band			
80-100 Pass (1st)	Fully implements all Basic, Medium, and Advanced Requirements of the Digital Library System. Includes complete CRUD operations, robust GUI, advanced validation, multi-threading, and advanced search/filtering features. The application demonstrates excellent usability, efficiency, and scalability.	The report is exceptionally well written and clearly structured. Provides deep analysis of the design and implementation, critically evaluates system strengths and limitations, and clearly justifies all design decisions using appropriate academic references.	Demonstrates outstanding professional software development practices. Code is clean, modular, and well-documented. Excellent use of version control with frequent commits, meaningful messages, and well-organised repository structure.
70-79 Pass (1st)	Successfully implements all Basic Requirements and most Medium Requirements, with partial implementation of Advanced features. The application is stable, user-friendly, and demonstrates effective GUI design and validation mechanisms.	The report is well organised and clearly explains the system design and development process. Provides good justification of design decisions and includes relevant academic references.	Strong adherence to coding standards and object-oriented principles. Regular use of version control with meaningful commit messages and clear project structure.
60-69 Pass (2.1)	Implements all Basic Requirements and most Medium Requirements. CRUD operations and GUI functionality work correctly. Some advanced features, such as enhanced validation or filtering, may be limited.	The report clearly explains system design and implementation. Provides reasonable justification of design decisions but may lack deeper critical analysis. Some referencing is included.	Demonstrates good coding practices and application of OOP concepts. Version control is used consistently, but commit messages may lack detail.
50-59 Pass (2.2)	Meets all Basic Requirements with some elements of Medium Requirements. CRUD functionality works, but the interface or validation features may be incomplete or inconsistent.	The report provides a general description of the development process but includes limited evaluation or critical discussion. Referencing may be minimal or inconsistent.	Basic coding standards and OOP concepts are demonstrated. Version control is used, but commits may be irregular or poorly described.
40-49 Pass (3rd) (Threshold)	Partially meets Basic Requirements. Some CRUD functionality may be incomplete or contain errors. System usability and stability may be limited.	The report provides a limited explanation of system design and implementation. The structure may be unclear and minimal. Referencing may be missing or inaccurate.	Basic attempt at coding standards and object-oriented structure. Minimal evidence of version control usage or poorly organised repository.
30-39 Fail	Fails to implement key Basic Requirements. Core functionality, such as CRUD operations or database integration, is incomplete or unreliable.	The report is poorly structured, incomplete, or lacks a meaningful explanation of the development process.	Limited evidence of coding standards or professional development practices. Little or no use of version control.
0-29 Fail	The application is non-functional or missing significant components. Required features are largely absent.	The report is missing or fails to address the assessment requirements.	No evidence of professional coding practices, object-oriented design, or version control usage.

CPS4005 - Object-Oriented Programming - Assessment Brief

Additional Artefacts and Guidance

Additional Artefact 1: Database File

You should utilise the following SQLite database:

```
-- Table to store books in the library
CREATE TABLE books (
  book_id INTEGER PRIMARY KEY,
  title TEXT NOT NULL,
  author TEXT NOT NULL,
  category TEXT NOT NULL,
  availability_status TEXT NOT NULL
);

-- Table to store library members
CREATE TABLE members (
  member_id INTEGER PRIMARY KEY,
  member_name TEXT NOT NULL,
  email TEXT NOT NULL,
  membership_type TEXT NOT NULL
);

-- Table to store borrowing transactions
CREATE TABLE borrow_records (
  record_id INTEGER PRIMARY KEY,
  book_id INTEGER NOT NULL,
  member_id INTEGER NOT NULL,
  borrow_date DATE NOT NULL,
  due_date DATE NOT NULL,
  return_status TEXT NOT NULL
);
```

CPS4005 - Object-Oriented Programming - Assessment Brief

Additional Artefact 2: Sample Data for Database File

You can populate the database with the following sample records:

-- Insert sample books

```
INSERT INTO books (title, author, category, availability_status)
VALUES
('Introduction to Java', 'John Smith', 'Programming', 'Available'),
('Database Systems', 'Maria Garcia', 'Computer Science', 'Borrowed'),
('Software Engineering Principles', 'Alan Brown', 'Engineering', 'Available');
```

-- Insert sample members

```
INSERT INTO members (member_name, email, membership_type)
VALUES
('Alice Johnson', 'alice.johnson@stmarys.ac.uk', 'Student'),
('Michael Lee', 'michael.lee@stmarys.ac.uk', 'Staff'),
('Sara Ahmed', 'sara.ahmed@stmarys.ac.uk', 'Student');
```

-- Insert sample borrowing records

```
INSERT INTO borrow_records (book_id, member_id, borrow_date,
due_date, return_status)
VALUES
(2, 1, '2025-03-01', '2025-03-15', 'Borrowed'),
(1, 2, '2025-03-02', '2025-03-16', 'Returned'),
(3, 3, '2025-03-05', '2025-03-19', 'Borrowed');
```

These records provide a starting point for testing the application and validating CRUD operations.

CPS4005 - Object-Oriented Programming - Assessment Brief

Additional Artefact 3: Book Record File Format

Below is an example format for a book record that may be used for import/export or display purposes.

BOOK ID: 1
TITLE: Introduction to Java
AUTHOR: John Smith
CATEGORY: Programming
STATUS: Available

Additional Artefact 4: Member Record File Format

A member record may follow the format below.

MEMBER ID: 1
MEMBER NAME: Alice Johnson
EMAIL: alice.johnson@stmarys.ac.uk
MEMBERSHIP TYPE: Student

Additional Artefact 5: Borrowing Record File Format

A borrowing transaction record may use the following format.

RECORD ID: 1
BOOK ID: 2
MEMBER ID: 1
BORROW DATE: 2025-03-01
DUE DATE: 2025-03-15
STATUS: Borrowed

CPS4005 - Object-Oriented Programming - Assessment Brief

Guidance on Database Usage

Students should consider the following when implementing the database:

Database Connectivity

Use the SQLite JDBC library to establish a connection between your Java application and the SQLite database.

Data Integrity

Ensure validation rules are applied before storing data in the database. Examples include:

- Book titles must not be empty
- Email addresses must follow valid formatting
- Borrow dates must be valid dates
- Due dates must be later than borrow dates

Error Handling

Your application should handle common database errors, such as:

- Invalid queries
- Database connection failures
- Constraint violations

Appropriate error messages should be displayed to the user.

CPS4005 - Object-Oriented Programming - Assessment Brief

Learning Outcomes

This assessment will enable students to demonstrate the following learning outcomes as stated in the module outline:

[Module Learning Outcome 2]

- Analyse a problem and design a solution using object-oriented principles while upholding appropriate ethical and professional standards

How is this learning outcome addressed?

Students are required to analyse the challenges associated with managing books, members, and borrowing records in a digital library system. They must design a software solution using object-oriented principles such as encapsulation and abstraction while ensuring the system follows professional coding practices and ethical considerations in software development.

[Module Learning Outcome 4]

- Apply object-oriented concepts, including encapsulation, abstraction, inheritance, and polymorphism, while providing a rationale for the design decisions made.

How is this learning outcome addressed?

Students must design and implement a Java-based application using object-oriented programming concepts. These principles should be reflected in the structure of classes representing entities such as books, members, and borrowing records. The report should clearly explain how object-oriented principles were applied and justify key design decisions.

[Module Learning Outcome 5]

- Develop object-oriented solutions by integrating industry-standard tools and techniques.

How is this learning outcome addressed?

Students will develop a functional software system using industry-standard tools such as Java, an integrated development environment (e.g., NetBeans), and SQLite for database management. The integration of these technologies demonstrates the ability to build practical software solutions used in real-world development environments.

CPS4005 - Object-Oriented Programming - Assessment Brief

[Module Learning Outcome 6]

- Apply exception handling techniques to write robust code that can handle unexpected errors.

How is this learning outcome addressed?

Students must implement appropriate exception handling mechanisms to manage errors that may arise from user input, database operations, or system processes. By handling exceptions effectively, the application will be more robust and capable of managing unexpected situations without crashing.

[Module Learning Outcome 7]

- Apply problem-solving skills to design and implement a software project using object-oriented programming principles and best practices.

How is this learning outcome addressed?

Students are required to design, develop, and test a complete object-oriented software solution for the digital library management scenario. This process requires identifying system requirements, structuring classes, implementing functionality, and applying best practices in software development to deliver a working application.

CPS4005 - Object-Oriented Programming - Assessment Brief

Regulations, Policies, and Guidelines

Guidance for online submissions

<https://www.stmarys.ac.uk/policies/online-submissions.aspx>

Academic Misconduct

Any submission must be students' own work and, where facts or ideas have been used from other sources, these sources must be appropriately referenced. Please find a link to the academic misconduct policy below:

<https://www.stmarys.ac.uk/policies/academic-regulations.aspx>

Ethics Policy

The work being carried out by students must be in compliance with the Ethics Policy. Where there is an ethical issue, as specified within the Ethics Policy, then students will need ethical approval prior to the start of the project. Please find a link to the ethics policy below:

<https://www.stmarys.ac.uk/research/students/ethical-review-process.aspx>

Extenuating Circumstances

The University's Extenuating Circumstances procedure helps students facing challenges in assessment submission. To request an extension or deferment, submit an EC application with evidence. Approved cases will not incur academic penalties. For longer-term issues, contact Student Services. Please find a link to the EC policy below:

<https://www.stmarys.ac.uk/policies/extenuating-circumstances.aspx>

Reassessment

If a student fails to meet the assessment criteria, they may be eligible for a reassessment. The reassessment will follow the same guidelines as the original assessment, requiring the student to reattempt the tasks outlined. Students should review any feedback provided and make improvements to meet the expected learning outcomes. In case of reassessment, it is important to consult with the module convenor for clarity on the areas of improvement and ensure compliance with submission deadlines and regulations.